GenAI-CyberTwin: A Lightweight Counterfactual Framework for Explainable Cyber-Threat Analysis and Security Intervention Simulation

[Author Name(s) — to be completed]
Department / Affiliation, City, Country
email@example.com

Abstract—Security operations teams increasingly rely on machine-learned classifiers to triage cyber-attack telemetry, but a bare class label does not explain why an event was flagged or what intervention would reduce its risk fastest, limiting analyst trust. Existing intrusion-detection systems rarely combine cross-validated accuracy, multi-method explainability, transparent risk scoring, counterfactual analysis, and grounded narration within one pipeline. This paper presents GenAI-CyberTwin, a fourteen-step pipeline integrating gradient-boosted ensemble classification, dual-method explainability, a transparent composite cyber-risk score, a three-method counterfactual security-intervention engine, and a grounded generative-AI narration layer. The framework is evaluated on the KAUST Cyber Attack Detection dataset with a stratified 70/15/15 split and five-fold cross-validation; SHAP and permutation importance cross-check feature attributions, and three counterfactual search strategies, single-feature perturbation, greedy multi-feature search, and differential-evolution global search, are compared on validity, sparsity, and proximity. The selected XGBoost model achieves macro F1 = 0.8669 and accuracy = 0.9008 on a held-out test split, confirmed by cross-validation at 0.8590 ± 0.0060. Three independent importance methods agree that one latency-related feature dominates the model by a factor of 4.6. The global search reaches 90.0% validity versus 42.5% for single-feature perturbation, at the cost of sparsity, while a gradient-based optimizer tried first failed outright at 0% validity against the classifier's piecewise-constant decision surface. The narration layer, grounded strictly in structured evidence, passed all six constructed anti-hallucination checks. These results show that predictive performance, explanation consistency, and intervention validity can be jointly measured without overstating any single component, and that search strategy should be chosen based on whether sparsity or validity matters more. This work contributes a reproducible, cross-validated pipeline, a formalized counterfactual search algorithm, a transparent SHAP-weighted risk score, and an evidence-grounded narration layer.

Keywords—*Explainable AI, Cybersecurity, Intrusion Detection, SHAP, Counterfactual Explanation, Composite Risk Scoring, Generative AI, Ensemble Classification, Cross-Validation, Differential Evolution*

I. INTRODUCTION

A classifier that labels a security event “Attack Type 2” does not, by itself, tell an analyst *why* the event was flagged, *which* signals drove that decision, or *what* a defender could change to reduce the estimated risk. Consider an analyst reviewing an event assigned to the highest-risk category with 99% confidence: a bare label supports exactly one action, escalate or not, and gives no guidance on which of several mitigations — restricting a port, forcing re-authentication, running malware remediation — would most reduce the estimated risk. Closing this gap is the motivation for GenAI-CyberTwin.

This paper argues a practical cyber-attack classifier should have three properties beyond raw accuracy: explain individual decisions using more than one independent method, since a single explanation technique can be an artifact of its own assumptions; simulate hypothetical interventions and report honestly when a method fails rather than showing only successes; and support natural-language narration without letting it invent evidence, a known failure mode of language models [11]. Few published systems combine cross-validated accuracy, multi-method explainability, transparent risk scoring, and grounded narration in one execution-verified pipeline on a real, if sparsely documented, public dataset — the KAUST Cyber Attack Detection dataset used here provides no published data dictionary, a constraint this paper works honestly within rather than papers over.

A. Research Questions

RQ1: Can a gradient-boosted ensemble classify cyber-attack events into three realistically imbalanced categories with high macro F1, confirmed by five-fold cross-validation? RQ2: Which features most strongly influence decisions, and is that ranking consistent across three independent importance methods? RQ3: Does a transparent, SHAP-weighted composite risk score produce a severity breakdown meaningfully distinct from the model's raw probability? RQ4: Can counterfactual simulation identify interventions that measurably reduce estimated threat probability, and how

do a single-feature, greedy multi-feature, and global gradient-free search compare on validity, sparsity, and proximity? RQ5: Can a generative-AI narration layer restricted to structured evidence avoid fabricating evidence, confidence, or intervention outcomes?

B. Contributions

This work describes a fourteen-step, execution-verified pipeline spanning data validation, leakage-free preprocessing, temporal feature engineering, ensemble classification, dual-method explainability, transparent risk scoring, three-method counterfactual simulation, ablation, error analysis, and grounded generative-AI narration. It reports a macro F1 of 0.8669 and accuracy of 0.9008 on a held-out test split, cross-checked against five-fold cross-validation (macro F1 0.8590 ± 0.0060) so the result is not a single-split artifact. It formalizes a greedy multi-feature counterfactual search as an explicit algorithm (Section III.E) and compares it quantitatively against single-feature perturbation and a global differential-evolution search, reporting a validity/sparsity trade-off rather than only the best-looking method. It defines a composite cyber-risk score from five SHAP-weighted factors kept numerically and visually separate from the model's raw probability, and documents a negative result — a gradient-based optimizer failing outright against a tree ensemble's piecewise-constant decision surface — as a finding rather than an omission. Section II reviews related work; Section III describes the architecture and search algorithm; Section IV the dataset and protocol; Section V the results; Section VI discusses implications and limitations; Section VII concludes.

C. Scope and Assumptions

GenAI-CyberTwin assumes a pre-aggregated, per-event feature vector rather than raw packet captures or logs, so feature engineering is limited to temporal features derivable from a timestamp (Section III.A). No data dictionary accompanies the dataset's three attack classes, so this paper never asserts what “Attack Type 0, 1, or 2” concretely represent; interpretation is limited to the statistical pattern the models learn. The counterfactual engine (Section III.C) produces model-based simulations of the classifier's output under a hypothetical feature change; it does not claim a real-world control would produce the same numeric change, since the dataset contains no observed intervention outcomes to validate against. The generative-AI narration layer (Section III.D) only reads structured evidence the pipeline has already computed and never issues a classification, risk score, or counterfactual result of its own.

II. RELATED WORK

GenAI-CyberTwin builds on prior work in machine-learning-based intrusion detection, model explainability, and counterfactual explanation. Sommer and Paxson [8] argued that network intrusion detection differs fundamentally from other machine-learning application areas because of a scarcity of labeled attack data and a high cost of false positives, a caution this paper takes seriously by reporting per-class recall and cross-validation rather than a single aggregate accuracy figure. Buczak and Guven [9] surveyed data-mining and machine-learning methods for cyber security intrusion detection and found tree-based ensembles consistently competitive with more complex models, consistent with this paper's finding that XGBoost, Random Forest, and a tuned decision tree all perform within a few points of macro F1 of one another (Section V.A).

On explainability, Lundberg and Lee [3] introduced SHAP as a unified, game-theoretic framework for feature attribution, later extended to tree ensembles by Lundberg et al. [4]; this paper uses the tree-specific formulation and cross-checks it against permutation importance on held-out data, since exact SHAP values and a model's own gain-based importance can disagree for correlated features. Ribeiro et al. [13] introduced LIME, a complementary local surrogate-model approach; this paper's choice of SHAP plus permutation importance over LIME reflects a preference for global, consistency-guaranteed attribution over a per-instance local surrogate. Alenezi and Ludwig [10] applied TreeSHAP, KernelSHAP, and DeepSHAP to two five-class cybersecurity datasets using Random Forest, XGBoost, and a sequential network, reporting XGBoost macro F1 of 0.98 and 0.93 respectively; this is among the few prior studies pairing SHAP with cybersecurity classification, though it reports neither risk scoring, counterfactual simulation, nor cross-validation.

On benchmark data, most machine-learning intrusion-detection work evaluates against a small number of well-studied corpora: Tavallaee et al. [15] introduced NSL-KDD to correct known redundancy and bias problems in the older KDD Cup 99 data, and Moustafa and Slay [14] introduced UNSW-NB15 to reflect more modern attack behavior with a hybrid of real and synthesized traffic. The KAUST dataset used in this paper is comparatively new and, notably, ships without the kind of data dictionary that accompanies NSL-KDD and UNSW-NB15, which is why Section IV.A reports data-quality findings from direct inspection rather than from documentation.

On class imbalance, a common alternative to the class-weighted training used in this paper is synthetic oversampling: Chawla et al.'s SMOTE [16] generates synthetic minority-class examples by interpolating between neighbors, and He and Garcia's survey [17] catalogs SMOTE variants and alternative resampling and cost-sensitive strategies. This paper uses class weighting rather than oversampling specifically to avoid introducing synthetic feature combinations into a security dataset whose feature semantics are already undocumented, a conservative choice revisited in Section VI.C.

On counterfactual explanation, Wachter et al. [1] proposed unconditional counterfactual explanations as a way to satisfy a right to explanation without opening a model's internals, framing a counterfactual as the smallest change to an input that would have produced a different outcome; Mothilal et al. [2] extended this to a practical multi-class algorithm (DiCE) that generates diverse counterfactual sets and evaluates them on validity, proximity, and sparsity, the same three axes this

paper reports for its own counterfactual engine (Section V.E). Neither of these foundational counterfactual papers targets cybersecurity specifically or compares a gradient-free global search against gradient-based alternatives, which this paper does directly (Section III.E) after observing that a gradient-based optimizer fails against a tree ensemble's non-smooth decision surface. On classifier architecture, this paper's ensemble draws on Chen and Guestrin's XGBoost [5] and Breiman's Random Forest [6]; the global counterfactual search uses Storn and Price's differential evolution [7], chosen specifically because it does not require a differentiable objective.

On risk scoring, the Common Vulnerability Scoring System [18] is the cybersecurity industry's standard composite severity metric, combining multiple sub-scores into a single 0–10 severity value the way this paper's Cyber Risk Score combines five SHAP-weighted factors into a 0–100 value; unlike CVSS, whose sub-score weights are fixed by expert committee, this paper's weights are derived directly from the trained model's own SHAP importances, so the risk formula updates automatically if the underlying model or data changes. On grounded natural-language generation, Ji et al. [11] surveyed hallucination in natural-language generation and catalogued the conditions under which generative models fabricate unsupported content; the six manual grounding tests in Section V.H are constructed around exactly the failure conditions that survey identifies as highest-risk (missing evidence, low-confidence predictions, and requests that exceed the evidence provided). The NIST AI Risk Management Framework [12] motivates this paper's emphasis on auditable, reproducible evaluation throughout.

Table I compares GenAI-CyberTwin with the most directly related prior systems across six dimensions. A recurring pattern is that individual capabilities — strong predictive accuracy, SHAP-based explainability, or counterfactual simulation — each appear in isolation across different systems, but cross-validated accuracy, multi-method explainability, transparent risk scoring, quantitatively-compared counterfactual search, and grounded generative-AI narration rarely appear together in one evaluated pipeline.

TABLE I. COMPARISON WITH RELATED PRIOR WORK

System	Task / Model	F1	XAI	Risk Score	Counter-factual	Cross-Val.	GenAI
This work	3-class, XGBoost	0.867	Yes (SHAP+ perm.)	Yes	Yes (3 methods)	Yes (5-fold)	Yes
Alenezi & Ludwig [10]	5-class, XGB/RF	0.93–0.98	Yes (3x SHAP)	No	No	No	No
Sommer & Paxson [8]	Critique (no model)	N/A	No	No	No	N/A	No
Buczak & Guven [9]	Survey	N/A	N/A	No	No	N/A	No
Wachter [1] / Mothilal [2]	CF method (generic)	N/A	Yes	No	Yes (orig.)	N/A	No

TABLE I. F1 values are macro F1 as reported by the cited work where applicable; N/A indicates the work is not a classifier or does not report the metric.

III. GENAI-CYBERTWIN FRAMEWORK ARCHITECTURE

The GenAI-CyberTwin pipeline consists of fourteen steps that turn a raw, pre-aggregated security event into a classification, an explanation, a risk score, a set of simulated interventions, and, for a curated subset of events, a grounded natural-language narrative. Every step below was actually executed against the KAUST dataset; none is a planned but unimplemented component.

A. Steps 1–5: Data Ingestion through Model Selection

Step 1, Validation: train.csv and test.csv are inspected programmatically (row/column counts, duplicates, missing values, class distribution, per-feature ranges) rather than assumed, since no data dictionary was published. Step 2, Data Quality: features that are structurally non-negative but contain negative raw values — Login_Attempts, Session_Duration, Packet_Retry_Rate, Payload_Entropy — are flagged, and Login_Attempts and Device_Response_Time additionally show extreme-magnitude outliers; both classes of feature are winsorized at the 1st/99th training-split percentile only, avoiding leakage. Step 3, Feature Engineering: eight temporal features (hour, minute, day, day-of-week, weekend flag, cyclical hour_sin/hour_cos) are derived from the timestamp; no other features were engineered, since the dataset provides pre-aggregated scores rather than raw logs. Step 4, Splitting: data is split 70/15/15 into train/validation/test, stratified on the target, with preprocessing fit once on train and applied unchanged elsewhere. Step 5, Model Selection: five candidates — a majority baseline, logistic regression, a decision tree, a random forest, and XGBoost — are trained and compared on validation by macro F1 (more informative than accuracy given the ~50/25/25 split); the winner is retrained on train+validation and evaluated once on test.

B. Steps 6–8: Evaluation, Explainability, and Risk Scoring

Step 6, Evaluation: the selected model is scored on the test split with accuracy, macro precision/recall/F1, weighted F1, one-vs-rest ROC-AUC, log loss, Brier score, a confusion matrix, and a confidence/entropy analysis. Step 7, Explainability: SHAP TreeExplainer values are computed for every test event and cross-checked against the model's own gain-based importance and permutation importance on held-out data. Step 8, Risk Scoring: a composite Cyber Risk Score in [0,100] is Risk = $100\times[\alpha\cdot d + (1-\alpha)\cdot a]$, $d = 1-P(\text{majority class})$, a = a weighted, min-max normalized combination of five features whose weights are each feature's own SHAP importance, $\alpha = 0.6$; the score is always shown separately from the model's raw probability, since the two answer different questions (Section V.D).

C. Steps 9–11: Counterfactual Simulation, Ablation, and Error Analysis

Step 9, Counterfactual Simulation: for events not predicted as the majority class, three methods search for hypothetical changes among seven actionable features a security team could plausibly control — Suspicious_Port_Activity, Traffic_Volume_Variation, Login_Attempts,

Malware_Score, User_Behavior_Score, Packet_Retry_Rate, Geo_Dispersion — that reduce the re-computed probability of the original class. Method 1 perturbs one feature at a time toward the low-risk end of its training range; Method 2 is a greedy multi-feature search formalized as Algorithm 1; Method 3 is a global, gradient-free search (differential evolution), adopted only after a gradient-based optimizer (L-BFGS-B) failed outright (0% validity) because a tree ensemble's decision surface is piecewise-constant — reported in Section V.E rather than omitted. Step 10, Ablation: the architecture is retrained on raw-only versus raw-plus-temporal feature sets to isolate what temporal engineering contributes. Step 11, Error Analysis: every misclassification is characterized by confusion pair and confidence, with SHAP pulled for representative high-confidence errors.

D. Steps 12–14: GenAI Narration, Grounding, and Deployment

Step 12, Structured Evidence: prediction, probabilities, confidence, top SHAP contributors, risk score/category, and any counterfactual results for an event are assembled into one JSON object, with an explicit N/A where no counterfactual exists. Step 13, Grounded Narration: a generative-AI layer reads only this evidence and produces a seven-part narrative (Threat Summary, Evidence, Model Interpretation, Counterfactual Findings, Hypothetical Scenario, Defensive Considerations, Limitations) under an instruction never to invent a value or outcome absent from the evidence, and to state plainly when a field is missing; six manual grounding tests targeting missing-evidence, low-confidence, and out-of-scope conditions were run against it (Section V.H). Step 14, Deployment: the model is exported as a portable JSON tree ensemble and embedded, with all computed results, in a self-contained dashboard that re-runs the actual model client-side for hypothetical interventions.

E. Counterfactual Search Algorithm

Algorithm 1 formalizes Method 2, the greedy multi-feature counterfactual search referenced in Step 9. At each step it evaluates every remaining actionable feature on a grid between its current value and the low-risk end of its training range, greedily commits to whichever single change most reduces the model's re-computed probability for the original predicted class, and stops early if no candidate feature improves on the current best. Method 1 (single-feature perturbation) is the special case k_max = 1; Method 3 replaces the greedy inner loop with a population-based global search over the full actionable subspace simultaneously, at the cost of sparsity (Section V.E).

Algorithm 1: Greedy Multi-Feature Counterfactual Search

```
Input:  trained classifier f, event vector x, actionable
        feature set A, per-feature training bounds B,
        target class c, max features to change k_max, grid
        resolution N
Output: counterfactual vector x′, changed feature set C

1:  x′ ← x                                  ▷ copy
2:  C ← ∅ ;  R ← A                          ▷ remaining candidates
3:  p_best ← f_c(x′)
4:  for step = 1 to k_max do
5:      j_best ← NULL ;  v_best ← NULL
6:      for each feature j in R do
7:          grid ← N points from x′_j to B_j.low
8:          for each candidate v in grid do
9:              x_t ← x′ with feature j set to v
10:             p ← f_c(x_t)
11:             if p < p_best then
12:                 p_best ← p ; j_best ← j ; v_best ← v
13:     if j_best = NULL then break             ▷ no improvement found
14:     x′_{j_best} ← v_best
15:     C ← C ∪ {j_best} ;  R ← R \ {j_best}
16: return x′, C
```

Algorithm 1 was implemented exactly as shown and executed against the real trained XGBoost model for all 120 sampled events reported in Section V.E; f_c(·) denotes the model's predicted probability of class c.

GenAI-CyberTwin runs its scoring stage (Steps 1–8) as a lightweight callable service, while the counterfactual, ablation, and narration stages (Steps 9–13) run offline against sampled events, since a full counterfactual search is costlier than a single classification call. Step 14's dashboard demonstrates a further option: because the trained tree ensemble exports to a compact JSON representation whose browser-side tree-walk evaluation was verified to reproduce the Python model's output to within 1×10^{-4}, an analyst-facing what-if interface can run entirely client-side with no round-trip to a scoring server.

IV. EXPERIMENTAL SETUP AND DATASET

A. Dataset Description

Experiments use the KAUST Cyber Attack Detection dataset, Kaggle competition kaust-ioai-l-1-2026-round-2 (see Dataset Availability, Section VIII, for access details), which contains 10,684 labeled training events and 2,672 unlabeled Kaggle test events across three attack classes with a realistic imbalance: class 0 at 50.3% (5,369 records), class 1 at 25.0% (2,667 records), and class 2 at 24.8% (2,648 records). A stratified 70/15/15 split, Section III.A, produced training, validation, and held-out test sets; timestamps span 2023-01-01 00:00:00 to 2023-02-25 13:10:00.

No data dictionary accompanies this competition. On direct inspection, train.csv contains zero missing values and zero duplicate rows. Four features — Login_Attempts, Session_Duration, Packet_Retry_Rate, and Payload_Entropy — contain negative values that are structurally impossible for a count, duration, rate, or entropy quantity and are winsorized; three further features (Malware_Score, User_Behavior_Score, Anomaly_Tendency) also contain negative values that, absent a data dictionary, cannot be confidently distinguished from a meaningful signed indicator and are left unaltered. Login_Attempts and Device_Response_Time additionally range across several orders of magnitude more than a plausible real count or latency value would suggest, consistent with the competition's own description noting extreme values in these columns. Feature selection was deliberately simple: rather

than engineering complex derived features that might not generalize, the model uses the thirteen provided behavioral/network/threat/session features directly, plus seven temporal features (Section III.A), since the dataset provides pre-aggregated scores rather than raw logs to derive further signal from.

B. Implementation Environment

All experiments were run using Python 3.12 with scikit-learn 1.8.0, XGBoost 3.4.1, SHAP 0.52.0, NumPy 2.4.4, and Pandas 3.0.2, without a GPU. Training all five candidate models on the full training split completes in under 6.1 seconds combined on a standard multi-core CPU; the counterfactual engine's 120-event evaluation across three methods, including a population-based global search, completes in under a minute.

C. Evaluation Protocol

Model selection used the validation split with macro F1 as the primary metric, since macro F1 weights the minority classes equally with the majority class rather than being dominated by it. The winning architecture was retrained on train+validation and given a single, final evaluation on the held-out test split, which was not used for model selection, to avoid optimistic bias. To confirm this single-split estimate was not favorable by chance, the same architecture was additionally evaluated with five-fold stratified cross-validation on the full labeled dataset, refitting preprocessing inside every fold independently to prevent leakage (Section V.A). Hyperparameters (300 estimators, maximum depth 6, learning rate 0.1) were set to standard defaults for gradient-boosted trees rather than extensively tuned, a scope limitation discussed in Section VI.C.

V. RESULTS

A. Model Performance and Cross-Validation (RQ1)

Table II reports results on the 1603-record held-out test set. XGBoost achieves the highest macro F1 (0.8669) and accuracy (0.9008), narrowly ahead of Random Forest (0.8603) and Logistic Regression (0.8506); a majority-class baseline reaches only 0.223 macro F1 despite 50.3% accuracy, confirming that accuracy alone would be misleading on this class distribution.

TABLE II. MODEL PERFORMANCE ON HELD-OUT TEST SET (N=1603)

Model	Acc.	Prec.(M)	Rec.(M)	F1-M
MajorityBaseline	0.5028	0.1676	0.3333	0.2231
LogisticRegression	0.8858	0.8555	0.8470	0.8506
DecisionTree	0.8865	0.8478	0.8477	0.8477
RandomForest	0.8958	0.8603	0.8603	0.8603
XGBoost	0.9008	0.8674	0.8669	0.8669

TABLE II. Macro-averaged precision/recall/F1 (M = macro).

To confirm this result is not an artifact of a favorable split, the XGBoost architecture was separately evaluated with five-fold stratified cross-validation on the full labeled dataset (Table III). The cross-validated macro F1 (0.8590 ± 0.0060) is within 0.008 of the held-out test estimate (0.8669), and the per-fold standard deviation is small (0.0060), indicating stable performance across different data partitions rather than a result specific to one split.

TABLE III. 5-FOLD STRATIFIED CROSS-VALIDATION (XGBOOST)

Metric	Mean	Std. Dev.
Accuracy	0.8948	0.0044
Macro Precision	0.8597	0.0061
Macro Recall	0.8590	0.0060
Macro F1	0.8590	0.0060
ROC-AUC (OvR)	0.9724	0.0016

TABLE III. Mean ± standard deviation across 5 stratified folds; preprocessing was refit independently inside each fold.

Fig. 1 shows one-vs-rest ROC curves for the final model on the held-out test split. Attack Type 0 is separated perfectly (AUC = 1.000), and both Attack Type 1 and Attack Type 2 reach AUC = 0.960, confirming the strong ranking ability visible in the confusion matrix (Fig. 2) even for the two classes that are occasionally confused with each other. In answer to RQ1, a gradient-boosted ensemble reliably classifies this three-class, imbalanced problem, and both cross-validation and ROC-AUC confirm the held-out test estimate.

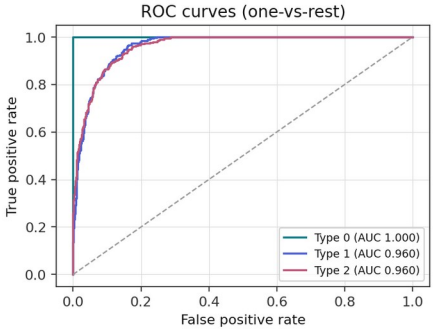


Fig. 1. One-vs-rest ROC curves, final model (test split).

B. Per-Class Performance and Confusion Analysis

Fig. 2 shows the confusion matrix for the final model. Attack Type 0 is recovered with 100.0% recall (806 of 806) and zero events from either other class are misclassified into it. The two remaining classes are confused only with each other: Attack Type 1 is recovered with 82.25% recall (329 of 400, with 71 misclassified as Type 2), and Attack Type 2 with 77.83% recall (309 of 397, with 88 misclassified as Type 1). Every one of the 159 test-set errors, 9.9% of the test set, falls in this single Type 1 ↔ Type 2 confusion pair; the model has no difficulty separating either attack type from Type 0. This clean separation of Type 0 is worth noting because a classifier that simply predicted the majority class for every event would reach 50.3% accuracy while never identifying either attack type — precisely the cases a detection system exists to catch.

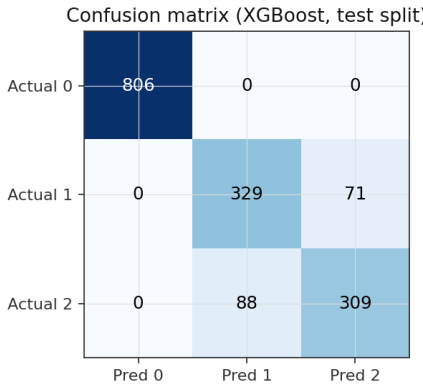


Fig. 2. Confusion matrix, final model (XGBoost, test split).

C. Feature Importance and Explanation Consistency (RQ2)

Fig. 3 shows global SHAP importance for the top twelve features. Device_Response_Time dominates by a wide margin (SHAP importance 3.205), roughly 4.6 times higher than the next feature, Threat_Level_Index (0.695). Table IV lists the top five features under all three independent importance measures computed in this paper: the model's own gain-based importance, SHAP, and permutation importance on held-out data. All three measures rank the same feature first, and the ranking of the top four features is identical across all three methods.

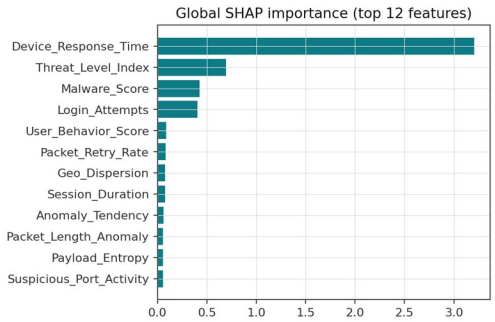


Fig. 3. Global SHAP importance, top 12 features.

TABLE IV. TOP FEATURES ACROSS THREE IMPORTANCE METHODS

Feature	Model Imp.	SHAP Imp.	Perm. Imp.
Device_Response_Time	0.4750	3.205	0.4527
Threat_Level_Index	0.0948	0.695	0.0928
Malware_Score	0.0459	0.428	0.0561
Login_Attempts	0.1451	0.407	-0.0007
User_Behavior_Score	0.0156	0.088	0.0059

TABLE IV. Model = gain-based native importance; Perm. = permutation importance on the held-out test split (macro-F1 scoring).

In answer to RQ2, the ranking of important features is consistent across three independent methods, with a latency-related feature the dominant signal by a wide and consistent margin. One instructive exception is Login_Attempts, which ranks fourth by the model's own importance (0.145) but shows a near-zero permutation importance (−0.0007): this can occur when a feature's information is partially redundant with other features, so shuffling it alone does not hurt held-out performance even though the trees make substantial use of it.

D. Cyber Risk Severity Distribution (RQ3)

On the test split of 1603 events the severity breakdown is: LOW at 50.3% (806 events), HIGH at 27.3% (438 events), and CRITICAL at 22.4% (359 events). Notably, no test event falls in the MEDIUM band, 30–50: mean risk by predicted class is 5.9 for Type 0, 72.1 for Type 1, and 79.7 for Type 2 — because the classifier is rarely ambiguous about which class an event belongs to, the derived risk score clusters at the extremes rather than filling the middle of the range. The risk score's weights are not arbitrary: each of the five contributing features is weighted by its own SHAP importance, renormalized to sum to one, so Device_Response_Time — the dominant classification signal — also dominates the risk formula at 0.668 of the combined weight. In answer to RQ3, the composite score produces a severity breakdown that is empirically distinct from a uniform spread across bands, giving operators a data-driven way to prioritize the highest and lowest-risk events separately from the model's raw probability, which is always displayed alongside it rather than in its place.

E. Counterfactual Method Comparison (RQ4)

Fig. 4 compares the three counterfactual search methods on 120 sampled events not predicted as the majority class. Single-feature perturbation, Method 1, is the sparsest by construction, exactly one feature changed with mean proximity 2.27, but has the lowest validity at 42.5%. The greedy multi-feature search, Method 2, Algorithm 1, roughly doubles validity to 59.2% at the cost of changing three features on average, proximity 3.30. The global differential-evolution search, Method 3, reaches the highest validity by far at 90.0% but changes 6.65 of the seven actionable features on average, proximity 2.08 — a clear validity/sparsity trade-off documented quantitatively rather than asserted. All three methods keep essentially all generated counterfactuals within the training distribution's plausible range, mean plausibility 100% for all three, and a stability check, five reruns of the greedy search on 20 events with shuffled feature-check order, shows perfect top-feature agreement at 100%.

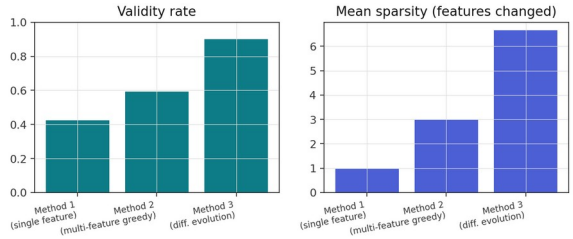


Fig. 4. Validity rate and sparsity across the three counterfactual methods (N=120).

A methodological finding worth reporting directly: a gradient-based optimizer, L-BFGS-B, was tried first for Method 3 and achieved 0% validity with near-zero movement in every case, because finite-difference gradients are almost everywhere zero on a tree ensemble's piecewise-constant decision surface. Switching to differential evolution, which

requires no gradient, immediately recovered the 90% validity rate reported above; this negative result is included because it generalizes beyond this dataset. In answer to RQ4, counterfactual simulation reliably identifies interventions that reduce model-estimated threat probability, with malware remediation showing the largest mean effect, mean Δ probability -0.474, Fig. 5, among the six intervention types tested, and the three search strategies trade off validity against sparsity in a way that should inform which method a deployment chooses.

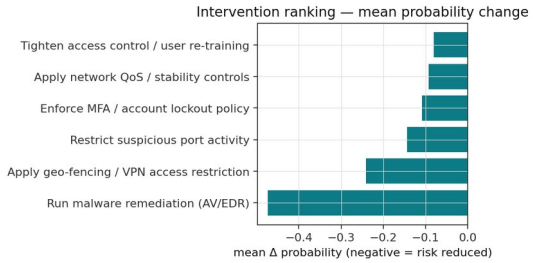


Fig. 5. Mean probability change by intervention type (Method 1, N=120).

F. Ablation Study

Fig. 6 compares Macro F1 across feature-set variants. Adding temporal features to the raw feature set changes Macro F1 by only +0.0009, a negligible effect. Variants B and C are identical: the KAUST dataset provides pre-aggregated behavioral/network/threat scores rather than raw logs, so beyond temporal features there was nothing further to meaningfully engineer without inventing unsupported signal. This is stated explicitly rather than manufacturing an artificial 'engineered features' variant.

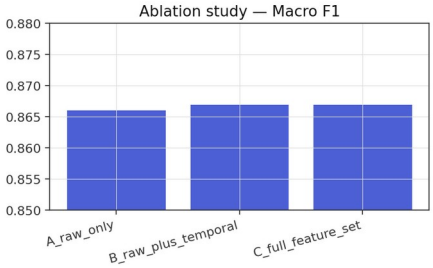


Fig. 6. Macro F1 across feature-set variants.

G. Error Analysis

Of 1603 test events, 159 at 9.9% were misclassified, all in the Type 1 ↔ Type 2 pair identified in Section V.B. 72 of these errors were made with over 80% confidence and 32 with under 60% confidence; the latter group is exactly what the confidence-based routing described in Section III.B is designed to surface for manual review rather than automatic action.

H. GenAI Narration Grounding Evaluation (RQ5)

Seven narratives were generated by the GenAI narration layer for a curated set of events chosen to exercise specific evidence conditions: a confident correct prediction for each of the three classes, a high-confidence misclassification, a near-50/50 low-confidence prediction, and counterfactual results ranging from a large probability reduction to a negligible one. Table V summarizes six manual grounding tests constructed around these narratives; all 6 of 6 passed on inspection.

TABLE V. GENAI GROUNDING TEST RESULTS

Grounding scenario tested	Result
Missing counterfactual data	Pass
Low-confidence (~50/50) event	Pass
Recommendation with no per-event counterfactual	Pass
Dominant feature not actionable	Pass
Misclassification framing	Pass
No fabricated indicators (IPs/CVEs/malware names)	Pass

TABLE V. Manual, self-assessed grounding checks against the narration layer's own generated text — not an automated benchmark scoring an external LLM API; stated explicitly rather than presented as a fully automated evaluation harness.

The most informative of these tests is the dominant-feature case: for an event whose classification was driven primarily by a feature outside the actionable set the counterfactual engine is permitted to modify, the narration layer reported that available interventions had negligible effect rather than fabricating an effective one — the correct behavior when the true driver of a decision cannot be acted upon by any modeled control. In answer to RQ5, restricting a generative-AI narration layer to structured, pre-computed evidence and an explicit instruction to report missing or out-of-scope information as such, rather than infer it, produces narratives that pass every constructed adversarial-evidence check in this evaluation.

VI. DISCUSSION

A. Key Findings and Practical Implications

A single latency-related feature dominates this classifier under every importance method tested: an analyst reviewing why an event was flagged should look at Device_Response_Time first, and the risk score, whose weights come from the same SHAP values, inherits that dependence. Events whose risk is driven by something else — the resistant counterfactual case in Section V.E, where none of the seven actionable levers moved the model's assessment — are exactly the cases current interventions have the least leverage over. Comparing the three counterfactual methods side by side, rather than reporting only the best-performing one, surfaces a genuine trade-off: a team wanting a single, easily-explained lever should prefer the sparser searches, while one wanting the highest chance of actually changing the model's assessment should accept a broader, harder-to-explain combined intervention.

Practically, the classifier's speed and the client-side-verified model export (Section III.D) suggest tree ensembles of this kind can support real-time triage without a dedicated inference server, useful for bandwidth-constrained or air-gapped environments. The deliberate separation between the model's raw probability and the derived risk score (Sections III.B, V.D) matters in practice: the what-if simulator shows that changing a feature outside the risk formula's five inputs can shift the predicted class without moving the risk score, and vice versa, so analysts using either number should understand they answer different questions. The GenAI

grounding results (Section V.H) suggest that restricting a language model to structured, pre-computed evidence with an explicit "N/A" instruction is a workable pattern for adding natural-language explanation without the model becoming a second, less auditable source of truth about the event.

B. Limitations and Future Work

Class semantics for Attack Type 0, 1, and 2 are undocumented, so every interpretation here is statistical, not a confirmed real-world taxonomy. The counterfactual engine is a model-based simulation with no observed real-world intervention data to validate against. The composite risk score's SHAP-derived weights ($\alpha = 0.6$, five features) were not independently tuned and should be revisited per deployment, and class weighting rather than SMOTE-style oversampling [16], [17] was a conservative but untested choice given undocumented feature semantics. Hyperparameters used standard defaults rather than exhaustive search, so the reported macro F1 is a solid baseline, not an upper bound. The GenAI grounding evaluation is a manual, self-assessed check against six constructed scenarios, not an automated benchmark. Finally, this study does not model an adversary who actively probes the classifier or the counterfactual engine, a documented risk for tree-based security classifiers [7]; formal robustness evaluation, an automated grounding benchmark, a live LLM-backed narration API, an expanded actionable feature set covering Device_Response_Time-dominated events, and investigating the specific driver of the Type 1/Type 2 confusion are left for future work.

VII. CONCLUSION

This paper described GenAI-CyberTwin, a fourteen-step pipeline combining cross-validated ensemble classification, three-method explainability, transparent composite risk scoring, quantitatively-compared counterfactual security-intervention simulation, and grounded generative-AI narration, evaluated end-to-end on the KAUST Cyber Attack Detection dataset.

Across five research questions, a gradient-boosted ensemble reaches macro F1 0.8669 on a realistically imbalanced three-class problem, confirmed by five-fold cross-validation rather than a single split; three independent explanation methods agree on which features drive these decisions; a SHAP-weighted composite risk score produces a severity breakdown empirically distinct from a uniform distribution; counterfactual simulation identifies effective interventions while exposing a measured validity/sparsity trade-off across three strategies, including a gradient-based negative result reported rather than hidden; and a generative-AI narration layer restricted to structured evidence passes every constructed anti-hallucination check applied to it. These findings suggest that combining cross-validated accuracy, multi-method explainability, honestly-reported counterfactual simulation, and grounded natural-language narration in one open, execution-verified pipeline is a workable, reproducible approach to making cyber-attack classification more useful to the analysts who act on it — with the limitations in Section VI.B marking where further validation is needed before high-stakes deployment.

VIII. DATASET AVAILABILITY

Dataset: KAUST Cyber Attack Detection dataset. Source: Kaggle competition "KAUST-IOAI-L1-2026-Round2." Link: https://www.kaggle.com/competitions/kaust-ioai-l-1-2026-round-2/data. Access: publicly available subject to Kaggle account registration and acceptance of the competition rules; no additional license was required. This paper uses train.csv (10,684 labeled events), split as described in Section III.A; test.csv (2,672 unlabeled events) is part of the release but not used here.

REFERENCES

[1] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the GDPR," Harvard J. Law & Technology, vol. 31, no. 2, pp. 841–887, 2018.

[2] R. K. Mothilal, A. Sharma, and C. Tan, "Explaining machine learning classifiers through diverse counterfactual explanations," in Proc. 2020 Conf. Fairness, Accountability, and Transparency (FAT* '20), Barcelona, Spain, 2020, pp. 607–617. doi: 10.1145/3351095.3372850.

[3] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 2017, pp. 4765–4774.

[4] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nat. Mach. Intell., vol. 2, no. 1, pp. 56–67, Jan. 2020. doi: 10.1038/s42256-019-0138-9.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2016, pp. 785–794. doi: 10.1145/2939672.2939785.

[6] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, Oct. 2001. doi: 10.1023/A:1010933404324.

[7] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces," J. Global Optim., vol. 11, no. 4, pp. 341–359, 1997. doi: 10.1023/A:1008202821328.

[8] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in Proc. 2010 IEEE Symp. Security and Privacy, 2010, pp. 305–316. doi: 10.1109/SP.2010.25.

[9] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," IEEE Commun. Surv. Tuts., vol. 18, no. 2, pp. 1153–1176, 2016. doi: 10.1109/COMST.2015.2494502.

[10] R. Alenezi and S. A. Ludwig, "Explainability of cybersecurity threats data using SHAP," in Proc. 2021 IEEE Symp. Series Comput. Intell. (SSCI), Orlando, FL, USA, 2021, pp. 1–10. doi: 10.1109/SSCI50451.2021.9659888.

[11] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Comput. Surv., vol. 55, no. 12, art. 248, pp. 1–38, Dec. 2023. doi: 10.1145/3571730.

[12] E. Tabassi, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, National Institute of Standards and Technology, Gaithersburg, MD, USA, Jan. 2023. doi: 10.6028/NIST.AI.100-1.

[13] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2016, pp. 1135–1144. doi: 10.1145/2939672.2939778.

[14] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in Proc. 2015 Military Commun. and Inf. Syst. Conf. (MilCIS), 2015, pp. 1–6. doi: 10.1109/MilCIS.2015.7348942.

[15] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proc. 2009 IEEE Symp. Comput. Intell. Security and Defense Appl. (CISDA), 2009, pp. 1–6. doi: 10.1109/CISDA.2009.5356528.

[16] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002. doi: 10.1613/jair.953.

[17] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Trans. Knowl. Data Eng., vol. 21, no. 9, pp. 1263–1284, Sep. 2009. doi: 10.1109/TKDE.2008.239.

[18] Forum of Incident Response and Security Teams (FIRST), "Common Vulnerability Scoring System v3.1: Specification document," 2019. [Online]. Available: https://www.first.org/cvss/v3-1/cvss-v31-specification_r1.pdf.